

REQUIREMENT ANALYSIS AND CUSTOMER DATABASE MODULE

1. Requirement Analysis

Understanding E-Commerce Business:

An e-commerce application is designed to support online purchasing by connecting customers, sellers, products, orders, and payment activities. A well-structured database is required to store these details and maintain the relationship between different activities of the purchasing process.

The Process Flow:

- **Browse / Search Product:** Customers can explore the available products and search for an item based on their requirements.
- **Add to Cart:** After selecting a suitable product, the customer can add it to the shopping cart.
- **Order Placed:** The customer provides the required delivery information and confirms the order.
- **Checkout / Payment:** The order moves to the checkout stage, where the customer completes the payment using an available payment method.
- **Delivery:** After successful payment, the order is processed for dispatch and delivered to the customer's specified address.
- **Review & Support:** Once the product is delivered, the customer can provide feedback or request assistance for returns, replacements, installation, or other product-related issues.

Entities:

1. **Customer:** The Customer entity stores the basic information of users who purchase products through the e-commerce platform. Each customer is identified by a unique customer ID.

Key Attributes:

Customer_ID (PK), Customer_Name, Email, Phone, created_at **Key**

Relationships:

One-to-Many relationship with Order and Review.

2. **Seller:** The Seller entity contains information about vendors who offer products on the platform. A seller may provide multiple products.

Key Attributes:

seller_id (PK), company_name, email, phone, rating, bank_details **Key**

Relationships:

One-to-Many relationship with Product.

3. **Product:** The Product entity represents the items listed by sellers for customers to purchase. It contains information related to the product, price, stock, and category.

Key Attributes:

product_id (PK), seller_id (FK), product_name, description, price, stock_quantity

Key Relationships:

Many-to-One relationship with Seller and One-to-Many relationship with Review.

4. **Order:** The Order entity maintains information about purchases made by customers. It records details such as the order date, status, amount, and delivery address.

Key Attributes:

order_id (PK), customer_id (FK), order_date, status, total_amount, shipping_address

Key Relationships:

Many-to-One with Customer, One-to-One with Payment, and Many-to-Many with Product through OrderItem.

5. **Payment:** The Payment entity records the transaction information associated with an order. It helps maintain payment method, transaction status, amount, and payment date.

Key Attributes: payment_id (PK), order_id (FK), payment_method, status, transaction_id, amount, payment_date

Key Relationships:

One-to-One relationship with Order.

6. **Review:** The Review entity maintains the feedback given by customers for products. It includes the rating and comments provided after the product experience.

Key Attributes:

review_id (PK), customer_id (FK), product_id (FK), rating, comment, created_at

Relationships:

Many-to-One relationship with Customer and Product.

2. Customer Table Creation

The Customer table is created to maintain the details of customers using the e-commerce application. The customer ID acts as the unique identifier, while the other fields store the customer's personal and registration information.

```
SQL> CREATE TABLE CUSTOMER(CUSTOMER_ID VARCHAR(7) PRIMARY KEY,CUSTOMER_NAME VARCHAR(106) NOT NULL, EMAIL VARCHAR(189) UNIQUE, PHONE NUMBER(15) UNIQUE, ADDRESS VARCHAR(290), REGISTRATION_DATE DATE NOT NULL);
Table created.
```

Insert Values:

Customer records are added to the database using the INSERT statement. Each record contains the required information corresponding to the columns defined in the Customer table.

```
SQL> INSERT INTO customer VALUES ('CUS001','Sanjay Kumar','sanjay@gmail.com',9876500001,'Chennai',DATE '2026-01-05');
1 row created.

SQL> INSERT INTO customer VALUES ('CUS002','Harini Devi','harini@gmail.com',9876500002,'Coimbatore',DATE '2026-01-08');
1 row created.

SQL> INSERT INTO customer VALUES ('CUS003','Praveen Raj','praveen@gmail.com',9876500003,'Madurai',DATE '2026-01-12');
1 row created.

SQL> INSERT INTO customer VALUES ('CUS004','Keerthana S','keerthana@gmail.com',9876500004,'Salem',DATE '2026-01-15');
1 row created.

SQL> INSERT INTO customer VALUES ('CUS005','Dinesh Kumar','dinesh@gmail.com',9876500005,'Trichy',DATE '2026-01-20');
1 row created.
```

After inserting the records, the Customer table can be displayed to verify the stored information.

```
SQL> SELECT * FROM CUSTOMER;
```

CUSTOMER_ID	CUSTOMER_NAME	EMAIL	PHONE	ADDRESS	REGISTRATION_DATE
CUS001	Sanjay Kumar	sanjay@gmail.com	9876500001	Chennai	05-JAN-26
CUS002	Harini Devi	harini@gmail.com	9876500002	Coimbatore	08-JAN-26
CUS003	Praveen Raj	praveen@gmail.com	9876500003	Madurai	12-JAN-26
CUS004	Keerthana S	keerthana@gmail.com	9876500004	Salem	15-JAN-26
CUS005	Dinesh Kumar	dinesh@gmail.com	9876500005	Trichy	20-JAN-26

Update:

The UPDATE statement is used to modify an existing customer's information. A particular customer can be selected using the customer_id, and the required field can then be changed.

```
SQL> UPDATE CUSTOMER SET PHONE = 9007654321 WHERE CUSTOMER_ID = 'CUS002';  
  
1 row updated.
```

Conclusion

The Customer Database Module establishes the basic structure required for managing customer information in an e-commerce system. The identified entities and their relationships provide a foundation for connecting customers with orders, products, payments, and reviews. The Customer table operations demonstrate how customer records can be created, stored, viewed, and updated within the database.